

Diseño y Programación de Software Multiplataforma
Técnico en Ingeniería en Computación

PROYECTO DE CATEDRA
Integración de Firebase en EmprendeLink Mobile
Etapa 3 – foro 2

Universidad Don Bosco, El Salvador.

EmprendeLink creado por Devsco

Integrantes:
Tito Mauricio Nochez Villagran - NV101005
Marvin Francisco Pérez Calderón - PC253641
Rafael Mena Mejia - MM253045
Luis Miguel Granados Artiga - GA130557
Jorge Alfonzo Mendoza Padilla - MP241100

Repositorio de GitHub: https://github.com/Notchez/emprendelink-dps.git

Link de video:
https://drive.google.com/file/d/193I_X1wIcjb5Sco1I5sHzGff7by8sHJP/view?usp=drive_link

Contenido

1. Introducción 3
2. Objetivos 4
3. Fundamentación teórica 4
4. Arquitectura de EmprendeLink 6
5. Integración de Firebase en Android 6
6. Autenticación con Google y correo/contraseña 7
7. Persistencia, restauración y recuperación de sesión 8
8. Modelo de datos de Cloud Firestore 10
9. Creación de perfiles y pedidos 11
10. Reglas de seguridad, autorización y protección de datos 12
11. Resultados de implementación y pruebas 14
12. Ventajas, limitaciones y decisiones técnicas 14
13. Conclusiones 15
Referencias 17

1. Introducción

El desarrollo de aplicaciones móviles conectadas exige resolver de forma coherente la identidad del usuario, la persistencia de datos y la protección de operaciones sensibles. EmprendeLink aborda estas necesidades mediante una arquitectura híbrida: la aplicación móvil está construida con React Native y Expo; Firebase Authentication gestiona la identidad; Cloud Firestore conserva perfiles, catálogos y datos transaccionales; y una aplicación Next.js expone Route Handlers para las operaciones privilegiadas. El propósito de esta investigación es explicar cómo esa integración fue implementada realmente en el repositorio y qué evidencia existe sobre su funcionamiento.

Firebase Authentication permite combinar credenciales de correo y contraseña con proveedores federados como Google, mientras que Cloud Firestore organiza la información en documentos agrupados en colecciones (Google, n.d.-b, n.d.-c, n.d.-d). En EmprendeLink, ambos servicios no se consumen de manera indiscriminada: las lecturas públicas y el perfil propio se atienden desde el SDK cliente, pero los pedidos, su historial y las comisiones pasan por el servidor. Esta separación responde a la necesidad de recalcular precios, comprobar roles y conservar datos económicos históricos sin confiar en valores enviados por el dispositivo.

La investigación se fundamenta en el código fuente, los contratos de datos, las reglas de Firestore y las pruebas automatizadas del repositorio. Se complementa con el recorrido funcional realizado en un Samsung Galaxy S23 Ultra, en el que se comprobó el acceso con Google, la creación del perfil, el registro de un pedido real y la restauración de la sesión después de Force Stop. La demostración audiovisual documenta este recorrido y permite relacionar los resultados con la arquitectura implementada.

2. Objetivos

2.1 Objetivo general

Analizar la integración de Firebase Authentication y Cloud Firestore en EmprendeLink Mobile, relacionando el diseño técnico del repositorio con la seguridad, persistencia de sesión, modelo de datos y resultados funcionales verificados en Android.

2.2 Objetivos específicos

- Describir el flujo de autenticación por correo/contraseña y Google, así como la creación del perfil asociado al UID de Firebase Authentication.
- Explicar la persistencia de sesión con AsyncStorage, la restauración del perfil y la recuperación ante fallos transitorios de Firestore.
- Representar las colecciones y relaciones principales de Firestore, con énfasis en usuarios, negocios, productos, pedidos, historial y comisiones.
- Evaluar las decisiones de autorización que separan el acceso directo del cliente de las operaciones privilegiadas ejecutadas mediante Next.js y Firebase Admin.
- Contrastar la implementación con los resultados de las pruebas funcionales y automatizadas, identificando sus ventajas y límites técnicos.

3. Fundamentación teórica

3.1 Firebase y arquitectura orientada a servicios

Firebase reúne servicios administrados para aplicaciones web y móviles. En este proyecto no se utiliza como un backend monolítico, sino como un conjunto de capacidades: Authentication emite la identidad y los tokens; Firestore almacena documentos; las Security Rules autorizan al SDK cliente; y Firebase Admin permite al servidor realizar acciones

privilegiadas. Esta composición reduce infraestructura propia, pero exige mantener fronteras claras entre configuración pública, credenciales privadas y validaciones de negocio.

3.2 Firebase Authentication

Firebase Authentication admite varios métodos de inicio de sesión y unifica la identidad resultante en una cuenta del proyecto. En el flujo federado, una aplicación puede obtener un ID token de Google y convertirlo en una credencial de Firebase; en el flujo tradicional, el SDK procesa correo y contraseña (Google, n.d.-b, n.d.-d). En ambos casos, el UID constituye el identificador estable que EmprendeLink utiliza como ID del documento en la colección users. La contraseña nunca se guarda en Firestore.

En React Native, la persistencia debe declararse mediante initializeAuth y getReactNativePersistence, enlazando un almacenamiento compatible con AsyncStorage (Google, n.d.-f). Async Storage es persistente y asíncrono, pero no cifra por sí mismo los valores almacenados (React Native Community, n.d.); por ello, EmprendeLink delega el estado de autenticación al SDK y no lo utiliza para almacenar contraseñas, tokens de servicio ni información económica.

3.3 Cloud Firestore

Cloud Firestore es una base NoSQL orientada a documentos: cada documento contiene pares clave-valor y pertenece a una colección (Google, n.d.-c). Este modelo permite que users, businesses, products y orders evolucionen de manera independiente, aunque la aplicación debe mantener las relaciones mediante identificadores. Firestore también ofrece transacciones atómicas, en las que todas las escrituras se confirman juntas o ninguna se aplica; este comportamiento es apropiado para crear un pedido y su historial de forma coherente (Google, n.d.-g).

4. Arquitectura de EmprendeLink

El repositorio sigue el flujo UI → Context/hooks → Services/Route Handlers → Firebase. Las pantallas móviles no importan Firebase directamente. AuthContext coordina la sesión; authService y userService encapsulan el SDK cliente; orderService obtiene un Firebase ID token y llama a la API; y los Route Handlers de Next.js validan sesión, rol, propiedad y datos antes de usar Firebase Admin. Los Route Handlers se implementan en archivos route.js dentro de app, conforme al modelo del App Router (Vercel, 2026).

Figura 1

Arquitectura aplicada de EmprendeLink



Nota. Elaboración propia a partir de docs/ARCHITECTURE.md y los servicios del repositorio (Notchez, 2026).

La primera ruta de la figura se utiliza para autenticación, perfiles y catálogo. La segunda protege la lógica central de pedidos. Las bibliotecas de servidor operan con privilegios administrativos y no quedan limitadas por Security Rules; por ello, la documentación oficial exige controlar su acceso mediante credenciales del entorno confiable e IAM (Google, n.d.-a, n.d.-e). En EmprendeLink, esa responsabilidad se materializa en requireSession, requireBusinessOwner, requireOrderAccess y prepareOrder.

5. Integración de Firebase en Android

La aplicación móvil utiliza Expo SDK 57, React Native 0.86.3 y Firebase JavaScript SDK 12.18.0. El identificador Android configurado es com.emprendelink.app. Google Sign-In requiere código nativo, por lo que se probó mediante un development build con expo-dev-client; este tipo de compilación funciona como una versión propia de Expo Go y permite incluir bibliotecas nativas y configuración personalizada (Expo, n.d.-b).

La configuración local lee variables EXPO_PUBLIC_FIREBASE_* y EXPO_PUBLIC_GOOGLE_WEB_CLIENT_ID desde mobile/.env, archivo excluido de Git. El archivo google-services.json también permanece ignorado. Para EAS, app.config.js acepta GOOGLE_SERVICES_JSON como archivo secreto. Expo advierte que las variables EXPO_PUBLIC_ se incorporan al bundle y no deben contener claves privadas; en cambio, los archivos secretos de Google Services deben administrarse fuera del repositorio (Expo, n.d.-a, n.d.-c). La Service Account usada por Firebase Admin está reservada al servidor y nunca forma parte del APK.

Fragmento 1. Inicialización persistente de Firebase Authentication

Ruta: mobile/src/lib/firebase/client.js

```
export function getFirebaseAuth() {
  const app = getFirebaseApp();
  try {
    return initializeAuth(app, { persistence: getReactNativePersistence(AsyncStorage) });
  } catch (error) {
    if (error?.code === "auth/already-initialized") return getAuth(app);
    throw error;
  }
}
```

El método crea una única instancia de Auth y conecta la persistencia de React Native con AsyncStorage. Si la instancia ya fue inicializada, reutiliza getAuth(app), evitando duplicados durante recargas del development build.

6. Autenticación con Google y correo/contraseña

authService ofrece login, register, loginWithGoogle, completeProfile y logout. El inicio por correo normaliza la dirección a minúsculas y delega la verificación de la contraseña a Firebase. El registro público crea primero la identidad y después el documento users/{uid}; userService restringe los roles permitidos a CUSTOMER y ENTREPRENEUR, de modo que ADMIN no puede originarse desde la interfaz pública.

Google Sign-In se carga desde el módulo nativo incluido en el development build. La aplicación comprueba Google Play Services, obtiene el ID token, construye una credencial con

GoogleAuthProvider y llama a signInWithCredential. Si la identidad aún no tiene perfil, resolveSession devuelve profileRequired: true y la interfaz solicita nombre, rol y datos correspondientes antes de habilitar el flujo de cliente.

Fragmento 2. Conversión del token de Google en sesión Firebase

Ruta: mobile/src/services/authService.js

```
const { GoogleSignin } = await loadGoogle();
await GoogleSignin.hasPlayServices({ showPlayServicesUpdateDialog: true });
const response = await GoogleSignin.signIn();
const idToken = response?.data?.idToken || response?.idToken;
if (!idToken) throw new Error("Google no devolvió un token de identidad.");
const googleCredential = dependencies.GoogleAuthProvider.credential(idToken);
const credential = await dependencies.signInWithCredential(
  dependencies.auth,
  googleCredential
);
return resolveSession(credential.user, dependencies);
```

El fragmento no almacena el token en Firestore ni lo registra en pantalla. Lo transforma en una credencial aceptada por Firebase Authentication y resuelve el perfil mediante el UID autenticado. Este patrón coincide con el flujo documentado para entregar un ID token de Google a Firebase (Google, n.d.-b).

Los errores de autenticación se convierten en mensajes en español mediante códigos como auth/invalid-credential, auth/network-request-failed y auth/too-many-requests. Esta capa mejora la experiencia, pero no sustituye las restricciones del servidor ni las reglas de Firestore. Ocultar opciones visuales tampoco se considera autorización.

7. Persistencia, restauración y recuperación de sesión

La persistencia fue validada en el Samsung Galaxy S23 Ultra: después de iniciar sesión con Google, cerrar la aplicación mediante Force Stop y abrirla de nuevo, Firebase restauró la identidad y la aplicación recuperó el perfil. AuthContext mantiene por separado identity —la cuenta validada por Firebase— y user —el documento de negocio leído desde Firestore—. Esta separación es importante porque una lectura temporalmente fallida del perfil no equivale a un logout.

Durante la restauración, onAuthStateChanged notifica la identidad. authService consulta users/{uid}; si Firestore devuelve USER_READ_ERROR, AuthContext conserva identity, elimina cualquier perfil incierto y activa sessionRecoverable. RootNavigator presenta un estado de error con la acción 'Intentar de nuevo'. Al reintentar, restoreSession vuelve a consultar el perfil actual.

Fragmento 3. Descarte de respuestas antiguas durante la restauración

Ruta: mobile/src/services/authService.js

```
async subscribe(listener) {
  const dependencies = await loadDependencies();
  let revision = 0;
  const unsubscribe = dependencies.onAuthStateChanged(
    dependencies.auth,
    async (identity) => {
      const currentRevision = ++revision;
      if (!identity) {
        return listener(
          success({
            identity: null,
            user: null,
            profileRequired: false,
          })
        );
      }
      const result = await resolveSession(identity, dependencies);
      if (currentRevision !== revision) return;
      const recoverableIdentity =
        result.error?.code === "USER_READ_ERROR"
          ? identity
          : null;
      listener(result, recoverableIdentity);
    }
  );
  return () => {
    revision += 1;
    unsubscribe();
  };
}
```

El contador revision invalida consultas anteriores cuando Firebase emite un estado más reciente. Así, una lectura lenta del perfil no puede volver a publicar una sesión después de un logout. Solo USER_READ_ERROR conserva temporalmente la identidad como condición recuperable.

Las pruebas móviles cubren restauración correcta, fallo transitorio, recuperación posterior, logout real y descarte de una consulta antigua. Esta solución es deliberadamente pequeña: no implementa reintentos infinitos ni confunde disponibilidad de red con credenciales inválidas.

8. Modelo de datos de Cloud Firestore

EmprendeLink utiliza colecciones de nivel superior y referencias por ID. El UID de Authentication identifica al usuario; businessId enlaza negocios, categorías, productos, pedidos y comisiones; y orderId relaciona el pedido con su historial y comisión. Algunos datos se duplican intencionalmente como instantáneas históricas para que una orden conserve nombres y valores aunque las entidades originales cambien.

Tabla 1

Colecciones y relaciones principales

Colección	Identificador / relación	Contenido principal	Control de acceso
users	UID de Firebase Auth	Perfil, rol, contacto, dirección, active	Propietario y ADMIN; creación pública sin ADMIN
businesses	id; ownerId → users	Nombre, slug, planId, estado	Lectura pública; escritura del propietario o ADMIN
plans	planId	Límite, commissionRate, active	Lectura pública; escritura ADMIN
categories	id; businessId → businesses	Nombre y estado	Lectura pública; escritura del propietario
products	id; businessId/categoryId	Nombre, precio, imagen, estado	Lectura pública; escritura del propietario
orders	orderId; customerId/businessId	Cliente histórico, ítems, subtotal, entrega, plan y tasa	Solo mediante Route Handlers/Firebase Admin
orderHistory	documento; orderId → orders	Transición, autor y fecha	Solo mediante Route Handlers/Firebase Admin
commissions	orderId	Negocio, tasa, monto, plan y fecha	Solo mediante Route Handlers/Firebase Admin

Nota. Elaboración propia a partir de docs/DATA_CONTRACTS.md y firestore.rules (Notchez, 2026).

Relaciones resumidas

Figura 2. Relaciones principales del modelo de datos



La relación es lógica: Firestore no aplica claves foráneas. Por esa razón, el servidor valida que negocio, producto, categoría y plan existan y correspondan entre sí antes de persistir operaciones sensibles.

9. Creación de perfiles y pedidos

9.1 Perfil asociado al UID

userService normaliza nombre, correo, teléfono, dirección e indicaciones de entrega. El documento se escribe en users/{uid}, evitando una colección separada para clientes. El correo se normaliza y la contraseña queda exclusivamente en Authentication. Las reglas comprueban que el UID de request.auth coincida con el documento y que el rol inicial pertenezca a CUSTOMER o ENTREPRENEUR.

9.2 Pedido validado por el servidor

La aplicación móvil envía businessId, productId, quantity, deliveryAddress y notes. No envía un customerId confiable, nombre de producto ni precio definitivo. apiClient adjunta el Firebase ID token como Bearer; el Route Handler POST /api/orders exige el rol CUSTOMER; y prepareOrder obtiene el perfil autenticado, lee los productos reales y recalcula los precios almacenados. De este modo se rechazan cantidades no enteras, productos duplicados, artículos inactivos o pertenecientes a otro negocio.

orderFirestoreService crea el pedido dentro de una transacción. Primero lee el negocio y el plan actual; valida que commissionRate sea un decimal entre 0 y 1; calcula ítems y subtotal; y escribe tanto orders como orderHistory. La atomicidad de Firestore evita que una parte quede guardada sin la otra (Google, n.d.-g).

Fragmento 4. Instantánea económica conservada en el pedido

Ruta: src/services/orderFirestoreService.js

```
const order = {
  id: orderRef.id,
  businessId: data.businessId,
  customerId: data.customerId,
  customer: data.customer,
  items,
  subtotal,
  deliveryAddress: data.deliveryAddress,
  notes: data.notes,
  status: ORDER_STATUS.PENDING,
  createdAt,
  commissionPlanId: planSnapshot.id,
  commissionPlanName:
    typeof plan.name === "string" && plan.name.trim() ? plan.name : planSnapshot.id,
  commissionRate,
};
```

El pedido conserva cliente, nombres y precios de productos, plan y tasa de comisión. Si el perfil, producto o plan cambia después, el documento histórico mantiene las condiciones utilizadas al confirmar la compra.

La comisión no se crea al registrar el pedido. Solo se genera cuando la transición alcanza DELIVERED. El documento commissions/{orderId} reutiliza la tasa almacenada en la orden y calcula amount = subtotal × rate; el ID del pedido evita comisiones duplicadas. CANCELLED no produce comisión.

10. Reglas de seguridad, autorización y protección de datos

firestore.rules utiliza rules_version = '2' y funciones signedIn, hasRole, admin y ownsBusiness. users permite lectura al propietario o ADMIN; la creación exige UID propio, rol público permitido y campos válidos; y la actualización del propietario queda limitada a name, phone, address y deliveryInstructions. role, active, email y createdAt no pueden modificarse desde ese formulario. El primer ADMIN se asigna manualmente en Firebase Console.

El catálogo —businesses, plans, categories y products— permite lectura pública porque no contiene contraseñas ni secretos. La escritura queda limitada por rol y propiedad. El comodín final deniega el resto del acceso directo, por lo que orders, orderHistory y commissions solo se administran desde el servidor. Las Security Rules son reglas de autorización, no filtros parciales

de consultas; una consulta debe cumplir completamente las condiciones definidas (Google, n.d.-e).

Firebase Admin opera en un entorno privilegiado y no aplica las reglas cliente. En consecuencia, los Route Handlers vuelven a validar token, perfil activo, rol, propiedad del negocio, acceso al pedido, estructura del cuerpo y transiciones permitidas. La Service Account se lee desde FIREBASE_SERVICE_ACCOUNT_JSON únicamente en el servidor. La documentación de Firebase insiste en proteger las claves de cuentas de servicio y utilizar entornos confiables (Google, n.d.-a).

10.1 Controles aplicados

- CUSTOMER puede confirmar pedidos y consultar únicamente sus propias órdenes; no cambia estados.
- ENTREPRENEUR accede solo a pedidos de su negocio, después de verificar ownerId.
- ADMIN posee funciones administrativas, pero no puede registrarse públicamente ni suplantar a un cliente comprador.
- El servidor ignora customerId, changedBy, precio y nombre de producto enviados por el dispositivo y deriva esos valores desde la sesión o Firestore.
- Los archivos .env, google-services.json y Service Account permanecen fuera del control de versiones. Las variables EXPO_PUBLIC_ se consideran configuración visible, no secretos.

11. Resultados de implementación y pruebas

La validación funcional se realizó en un Samsung Galaxy S23 Ultra mediante un development build de Expo SDK 57. Se comprobó el inicio de sesión con Google, la creación del perfil de cliente, la escritura de un pedido real y la restauración de sesión después de Force Stop.

El documento de la orden en Firestore conservó los datos del cliente, el negocio, los productos, la dirección de entrega, el plan y la tasa de comisión. La demostración audiovisual complementa estos resultados.

Tabla 2

Resultados de las verificaciones funcionales

Funcionalidad	Tipo de evidencia	Resultado / alcance
Google Sign-In	Prueba manual en Samsung	Comprobado
Creación de perfil CUSTOMER	Prueba manual + Firestore	Comprobado con documento real
Pedido y datos económicos	Prueba manual + Firestore	Comprobado con orden real
Persistencia tras Force Stop	Prueba manual	Comprobado
Correo/contraseña	Código + pruebas automatizadas	Implementado y cubierto por pruebas automatizadas
Recuperación ante fallo de perfil	Pruebas automatizadas	Cubierta: error, reintento y recuperación
Autorización de pedidos	12 pruebas web	Sesión, rol, propiedad y datos forjados
Servicios móviles	40 pruebas móviles	Auth, sesión, catálogo, carrito y pedidos
Bundle Android	expo export	990 módulos; exportación correcta
CI del PR #13	GitHub Actions	Web y móvil completados correctamente

12. Ventajas, limitaciones y decisiones técnicas

12.1 Ventajas

- Una identidad única enlaza Authentication y users/{uid}, simplificando la asociación de datos al usuario.
- La capa de servicios mantiene Firebase fuera de los componentes visuales y facilita pruebas con dependencias inyectadas.
- Las operaciones económicas se validan en el servidor y conservan instantáneas históricas de precios, cliente, plan y comisión.
- Las transacciones mantienen coherencia entre pedido, historial y comisión, incluso ante concurrencia.
- El estado recuperable diferencia un fallo temporal de Firestore de un cierre de sesión auténtico.

12.2 Limitaciones actuales

- La validación manual documentada corresponde a Android y al rol CUSTOMER; no demuestra el mismo flujo en iOS ni todos los recorridos de ADMIN y ENTREPRENEUR en móvil.
- Las consultas de catálogo y pedidos son lecturas bajo demanda; no existe un listener en tiempo real para órdenes o historial.
- AsyncStorage es persistente pero no cifrado por sí mismo; su uso se limita a la persistencia administrada por Firebase Auth.
- El development build verifica la integración nativa, pero no sustituye una validación de un build de producción firmado y distribuido.

12.3 Decisiones técnicas defendibles

Se eligió el Firebase Web SDK en React Native para compartir conceptos y contratos con la aplicación web, manteniendo Firebase Admin únicamente en Next.js. También se conservaron JavaScript, Context API y los servicios existentes, evitando migraciones no requeridas. Para corregir la regresión de Metro asociada a import() dinámicos sin Expo Router, los cargadores de Firebase y Google Sign-In utilizan require() sincrónico, sin agregar dependencias.

13. Conclusiones

La integración de Firebase en EmprendeLink no se limita a conectar un SDK. El resultado combina identidad, perfil, reglas, servicios móviles, API y operaciones administrativas en capas diferenciadas. Firebase Authentication aporta el UID y el token; Firestore conserva datos documentales; y Next.js protege los procesos que no deben confiar en el dispositivo.

La evidencia disponible confirma el flujo principal de cliente en Android: autenticación con Google, creación del perfil, pedido real y persistencia después de Force Stop. Las pruebas

automatizadas amplían la cobertura hacia correo/contraseña, errores transitorios, logout, autorización por roles, protección contra datos forjados y exportación Android. Esta combinación de prueba manual y automatizada fortalece la trazabilidad sin exagerar lo observado.

La integración combina autenticación robusta, persistencia estructurada, autorización por reglas y validaciones del servidor. El diseño permite que el flujo móvil y la aplicación web compartan información coherente sin delegar decisiones económicas críticas al dispositivo. Entre las oportunidades de evolución se encuentran el soporte funcional de más roles en Android, la validación del recorrido en iOS y la incorporación de actualizaciones en tiempo real.

Referencias

Expo. (s. f.-a). Environment variables in Expo. Expo Documentation. https://docs.expo.dev/guides/environment-variables/

Expo. (s. f.-b). Introduction to development builds. Expo Documentation. https://docs.expo.dev/develop/development-builds/introduction/

Expo. (s. f.-c). Manage environment variables. Expo Documentation. https://docs.expo.dev/eas/environment-variables/manage/

Google. (s. f.-a). Add the Firebase Admin SDK to your server. Firebase Documentation. https://firebase.google.com/docs/admin/setup

Google. (s. f.-b). Authenticate using Google with JavaScript. Firebase Authentication. https://firebase.google.com/docs/auth/web/google-signin

Google. (s. f.-c). Cloud Firestore data model. Firebase Documentation. https://firebase.google.com/docs/firestore/data-model

Google. (s. f.-d). Get started with Firebase Authentication on websites. Firebase Documentation. https://firebase.google.com/docs/auth/web/start

Google. (s. f.-e). Get started with Cloud Firestore Security Rules. Firebase Documentation. https://firebase.google.com/docs/firestore/security/get-started

Google. (s. f.-f). JavaScript API reference: Authentication. Firebase Documentation. https://firebase.google.com/docs/reference/js/auth

Google. (s. f.-g). Transactions and batched writes. Firebase Documentation. https://firebase.google.com/docs/firestore/manage-data/transactions

Notchez. (2026). EmprendeLink DPS [Código fuente]. GitHub. https://github.com/Notchez/emprendelink-dps

React Native Community. (s. f.). Async Storage. https://react-native-async-storage.github.io/

Vercel. (2026, 27 de febrero). Route Handlers. Next.js Documentation. https://nextjs.org/docs/app/getting-started/route-handlers